GUION DE EXPOSICION AL PROFESOR - PROYECTO TIENDACOCIS (SH

Enfoque principal: Backend en Go (Golang), Arquitectura Hexagonal y Patrones Singleton y Strategy

1. INTRODUCCION Y CONTEXTO DEL PROYECTO (Minuto 1)

[Que decir al profesor]:

"Profesor, nuestro proyecto es TiendaCocis, orientado a la gestion de productos e insumos de embalaje de Shalom en Soles Peruanos (S/). El sistema maneja dos CRUDs principales en PostgreSQL (productos y almacen) ejecutados en Docker. En el Backend desarrollado en Go aplicamos de forma estricta camelCase, funciones en espanol de 1 a 3 palabras, Arquitectura Hexagonal y dos patrones de diseno: el Patron Creacional Singleton y el Patron de Comportamiento Strategy."

Estructura de Tablas en el Dominio (camelCase):

```
- Tabla productos : idProducto (PK), nombreProducto, precioUnidad, categoria
- Tabla almacen   : idInventorio (PK), idProducto (FK), stock, stockMin, ultimaActualizacion
```

2. ARQUITECTURA HEXAGONAL: PUERTOS Y ADAPTADORES EN GO (Minutos 2 y 3)

[Que decir al profesor]:

"Para desacoplar la logica de negocio de la base de datos y de la API REST, estructuramos el backend en Go dentro de la carpeta internal/ siguiendo la Arquitectura Hexagonal en 4 capas:"

A) Capa de Dominio (internal/domain/):

Archivos: product.go, inventory.go e inventoryStrategy.go.
Contiene las estructuras puras en camelCase y la logica matematica de las estrategias. No conoce nada de SQL ni de HTTP.

B) Capa de Puertos (internal/ports/):

- inbound.go (Puertos de Entrada): Interfaces ProductService e InventoryService con los metodos que los controladores HTTP pueden invocar (ListarProductos, CrearProducto, ActualizarStock...).
- outbound.go (Puertos de Salida): Interfaces ProductRepository e InventoryRepository que definen como el nucleo guarda o consulta datos sin atarse a un motor de base de datos especifico.

C) Casos de Uso / Core (internal/core/):

Archivos: productService.go e inventoryService.go.
Implementan los puertos de entrada, validan las reglas de negocio (precios positivos, existencia del producto antes de crear su stock) y aplican la estrategia de inventario correspondiente.

D) Capa de Adaptadores (internal/adapters/):

- Adaptadores Primarios (inbound/productHandler.go e inventoryHandler.go): Controladores HTTP que reciben las peticiones REST desde React + Next.js y devuelven JSON en camelCase.
- Adaptadores Secundarios (outbound/database.go, productRepository.go, inventoryRepository.go): Ejecutan las consultas SQL reales sobre PostgreSQL utilizando la conexion Singleton.

PATRONES DE DISENO EN EL BACKEND Y DEFENSA TECNICA

3. PATRON CREACIONAL: SINGLETON CON sync.Once (Minuto 4)

Archivo a mostrar al profesor: internal/adapters/outbound/database.go

[Que decir al profesor]:

"Aplicamos el Patron Singleton en DatabaseConnection porque abrir una conexion nueva a PostgreSQL en cada peticion HTTP consume demasiados recursos. Con sync.Once en la funcion ObtenerInstancia(), Go garantiza que la primera peticion cree la conexion y todas las demas reutilicen esa misma instancia unica en memoria de forma segura ante concurrencia."

```
var instancia *DatabaseConnection
var unaVez sync.Once
func ObtenerInstancia() *DatabaseConnection {
    unaVez.Do(func() { instancia = &DatabaseConnection{...} })
    return instancia
}
```

4. PATRON DE COMPORTAMIENTO: STRATEGY EN INVENTARIO (Minuto 5)

Archivo a mostrar al profesor: internal/domain/inventoryStrategy.go

[Que decir al profesor]:

"Para el calculo de valor del almacen y actualizacion de stock implementamos el Patron Strategy mediante la interfaz InventoryStrategy. Esto permite intercambiar dinamicamente tres reglas de negocio desde el selector del frontend sin modificar el codigo del servicio de inventario:"

1. **StandardStrategy (Estandar):** Calculo directo sin alteraciones (stock * precioUnidad).
2. **LiquidacionStrategy (Liquidacion):** Aplica 30% de depreciacion sobre el precioUnidad en almacen.
3. **VolumenStrategy (Por Volumen):** Bonifica +10% en stock y ajuste de valor en lotes >= 20 uds.

5. FLUJO COMPLETO Y PREGUNTAS FRECUENTES DEL PROFESOR

P1: ¿Donde se conectan los Puertos y los Adaptadores?
R: En cmd/server/main.go dentro de la funcion iniciarServidor(). Alli se instancia el Singleton de PostgreSQL, se pasa a los repositorios (outbound), luego a los servicios (core) y por ultimo a los controladores HTTP (inbound).

P2: ¿Por que usar Arquitectura Hexagonal en un proyecto de dos tablas?
R: Porque permite probar las reglas de negocio (como las estrategias del almacen) sin levantar la base de datos, y si en el futuro cambiamos PostgreSQL por otro motor SQL o NoSQL, el dominio y los casos de uso permanecen intactos.

P3: ¿Como se cumplio la regla de nombres de funciones en el backend?
R: Todas las funciones y metodos estan escritos en espanol, en camelCase y tienen entre 1 y 3 palabras como maximo: listarProductos, crearProducto, actualizarStock, responderJSON, etc.